

LAB_ASSIGNMENT – 19_1

NAME: G HASINI

BATCH:02

HT NO: 2303A51099

Task 1 – Python to Java Conversion

Translate a simple Python class into Java using AI assistance.

Instructions:

- Prompt AI to convert a Python class with attributes and methods into equivalent Java code.
- Verify that constructors, method signatures, and data types are properly handled.
- Run both versions (Python & Java) to compare expected output

CODE:

```
#write a python program to create proper classes and objects
class brother:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")
try:
    b1 = brother("John", 25)
    b2 = brother("Mary", 30)
    b1.display()
    b2.display()
except Exception as e:
    print(e)
```

#RUN THE above code in the java

```
class brother:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")

try:
    b1 = brother("John", 25)
    b2 = brother("Mary", 30)
    b1.display()
    b2.display()
except Exception as e:
    print(e)

public class Brother {
    private String name;
    private int age;

    public Brother(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }

    public static void main(String[] args) {
        try {
            Brother b1 = new Brother("John", 25);
            Brother b2 = new Brother("Mary", 30);
            b1.display();
            b2.display();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\GVN\AIAC\lab_19> & c:/python314/python.exe c:/Users/chvis/OneDrive/GVN/AIAC/lab_19/2.py
Name: John, Age: 25
Name: Mary, Age: 30
PS C:\Users\chvis\OneDrive\GVN\AIAC\lab_19>
```

Task 2 – C++ to Python Function Conversion

Task:

Convert a C++ function for factorial calculation into Python.

Instructions:

- Ask AI to translate a recursive C++ factorial function into Python.
- Validate correctness by testing with sample inputs.
- Ensure Python version follows Pythonic conventions.

Expected Output:

- A correct Python implementation equivalent to the original

C++ factorial function

CODE:

```
41 #write a c++ code function that calculates the factorial of a number
42 #include <iostream>
43 using namespace std;
44 // Function to calculate factorial
45 unsigned long long factorial(int n) {
46     if (n < 0) {
47         throw invalid_argument("Factorial is not defined for negative numbers.");
48     }
49     unsigned long long result = 1;
50     for (int i = 1; i <= n; ++i) {
51         result *= i;
52     }
53     return result;
54 }
55 int main() {
56     int number;
57     cout << "Enter a number to calculate its factorial: ";
58     cin >> number;
59     try {
60         unsigned long long fact = factorial(number);
61         cout << "Factorial of " << number << " is " << fact << endl;
62     } catch (const invalid_argument& e) {
63         cout << "Error: " << e.what() << endl;
64     }
65     return 0;
66 }
```

```
67 #write above into python code
68 def factorial(n):
69     if n < 0:
70         raise ValueError("Factorial is not defined for negative numbers.")
71     result = 1
72     for i in range(1, n + 1):
73         result *= i
74     return result
75 try:
76     number = int(input("Enter a number to calculate its factorial: "))
77     fact = factorial(number)
78     print(f"Factorial of {number} is {fact}")
79 except ValueError as e:
80     print(f"Error: {e}")
81
```

OUTPUT:

```
Enter a number to calculate its factorial: 10
Factorial of 10 is 3628800
```

Task 3 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

CODE:

```
82 #write a sql commands that create a table named as employee and insert some data and retrieve the same and salary which are salary greater than 50000
83 -- Create the employee table
84 CREATE TABLE employee (
85     id INT PRIMARY KEY,
86     name VARCHAR(50),
87     position VARCHAR(50),
88     salary DECIMAL(10, 2)
89 );
90 -- Insert some data into the employee table
91 INSERT INTO employee (id, name, position, salary) VALUES
92 (1, 'John Doe', 'Software Engineer', 60000.00),
93 (2, 'Jane Smith', 'Data Analyst', 45000.00),
94 (3, 'Emily Davis', 'Project Manager', 75000.00),
95 (4, 'Michael Brown', 'Intern', 30000.00);
96
97 -- Retrieve employees with salary greater than 50000
98 SELECT * FROM employee WHERE salary > 50000;
99
```

```
100 #write the above sql commands into python using pandas dataframe
101 import pandas as pd
102 # Create a DataFrame to represent the employee table
103 data = {
104     'id': [1, 2, 3, 4],
105     'name': ['John Doe', 'Jane Smith', 'Emily Davis', 'Michael Brown'],
106     'position': ['Software Engineer', 'Data Analyst', 'Project Manager', 'Intern'],
107     'salary': [60000.00, 45000.00, 75000.00, 30000.00]
108 }
109 employee_df = pd.DataFrame(data)
110 # Retrieve employees with salary greater than 50000
111 high_salary_employees = employee_df[employee_df['salary'] > 50000]
112 print(high_salary_employees)
113
```

OUTPUT:

	id	name	position	salary
0	1	John Doe	Software Engineer	60000.0
2	3	Emily Davis	Project Manager	75000.0

Task 4 – Real-Time Application: Multi-Language Snippet Converter

Scenario:

Students will choose a real-time use case where they need code in multiple languages. Example:

- Converting a Python web scraping snippet into JavaScript for browser automation.
- Translating a Java method into C# for enterprise applications.

Instructions:

- Prompt AI to translate code between two chosen languages.
- Test both implementations to ensure they produce the same results.
- Document challenges faced during translation (e.g., syntax, libraries).

Expected Output:

- Working equivalent code snippets in at least two different programming languages.

CODE:

```
def fibonacci(numbers):
    if numbers <= 0:
        return []
    elif numbers == 1:
        return [0]
    elif numbers == 2:
        return [0, 1]

    one_unique = 0
    two_unique = 1
    fib_series = [one_unique, two_unique]

    for _ in range(2, numbers):
        next_unique = one_unique + two_unique
        fib_series.append(next_unique)
        one_unique = two_unique
        two_unique = next_unique
    return fib_series

try:
    numbers = int(input("Enter the number of terms: "))
    fib_series = fibonacci(numbers)
    print("Fibonacci series up to", numbers, "terms:")
    print(fib_series)
except ValueError:
    print("Please enter a valid integer.")
```

```

import java.util.ArrayList;
import java.util.InputMismatchException;
import java.util.List;
import java.util.Scanner;

public class Main {
    public static List<Integer> fibonacci(int numbers) {
        List<Integer> fibSeries = new ArrayList<>();

        if (numbers <= 0) {
            return fibSeries;
        } else if (numbers == 1) {
            fibSeries.add(0);
            return fibSeries;
        } else if (numbers == 2) {
            fibSeries.add(0);
            fibSeries.add(1);
            return fibSeries;
        }

        int oneUnique = 0;
        int twoUnique = 1;
        fibSeries.add(oneUnique);
        fibSeries.add(twoUnique);

        for (int i = 2; i < numbers; i++) {
            int nextUnique = oneUnique + twoUnique;
            fibSeries.add(nextUnique);
            oneUnique = twoUnique;
            twoUnique = nextUnique;
        }

        return fibSeries;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        try {
            System.out.print("Enter the number of terms: ");
            int numbers = scanner.nextInt();
            List<Integer> fibSeries = fibonacci(numbers);
            System.out.println("Fibonacci series up to " + numbers + " terms:");
            System.out.println(fibSeries);
        } catch (InputMismatchException e) {
            System.out.println("Please enter a valid integer.");
        } finally {
            scanner.close();
        }
    }
}

```

OUTPUT:

```

Enter the number of terms: 25
Fibonacci series up to 25 terms:
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946, 17711, 28657, 46368]
PS C:\Users\chvis\OneDrive\GWN\AIAC\lab_19> & c:/python314/python.exe c:/Users/chvis/OneDrive/GWN/AIAC/lab_19/1.py
Enter the number of terms: 5
Fibonacci series up to 5 terms:
[0, 1, 1, 2, 3]

```